COMP6080

💥 ReactJS - Assignment 3 Intro & Demo

Lecture 8.1

Author(s): Hayden Smith

(Download as PDF)

🍔 Assignment 3

Let's take a look at together.

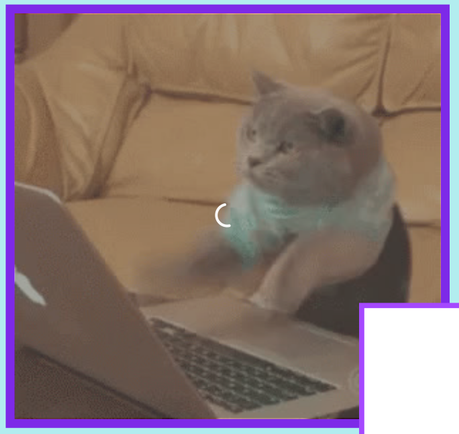
Your own personal copy of this repository has been deployed to your gitlab space.

Things we will consider:

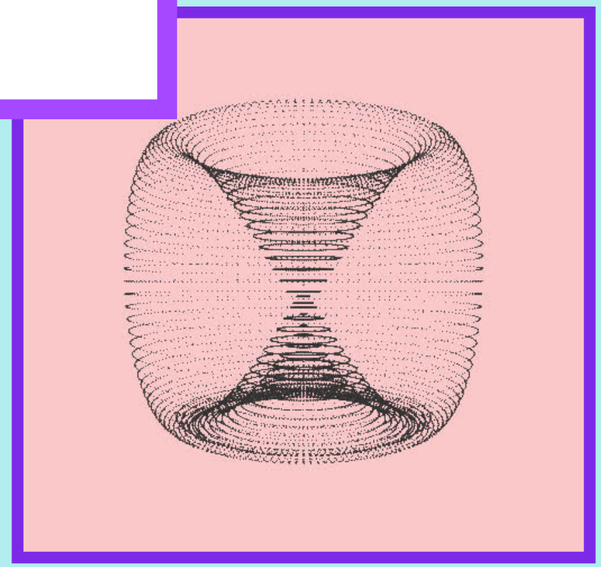
- Linting
- General Structure
- API Calls

👂 Feedback

Or go to the form here.



Linting



Presented by
Tom Isles

Canva

What is Linting?

Linting is a method of static code analysis.

It takes the code you write as an input, and analyses it in place to find errors.

It does not attempt to compile or run your code.

What is Linting?

It allows you to avoid common code mistakes or deviations from best practice - things that you might otherwise forget about.

It's fast. If properly integrated into your editor, it can report errors as soon as they are made.

It boosts productivity by providing a guardrail against human error.

Linting holds your hand while you code, and lets you focus on the big picture.

Linting is Not

Linting is not a substitute for good testing.

It is not a way to ensure your design is correct.

It is not a way to enforce a good architecture or system.

It is not a magic 'make my code work' tool.

It is not a system for enforcing the correct types.

It is not the only method of static code analysis (it is one of the most prevalent however)

A Bit of History...

Javascript is a language with a lot of rough edges. The first implementation was completed in 10 days for Netscape Navigator.

Initially, Javascript's creator (Brendan Eich) had a totally different language in mind (Scheme) but higher-ups at Netscape wanted a language that looked more like Java.

Over the years, Javascript has accumulated a lot of features that you should probably never use - in the early days, people created unmaintainable, buggy code with these features.

Douglas Crockford - Javascript: The Good Parts (2008)

A Bit of History...

In 2008, Douglas Crockford released *Javascript: The Good Parts*, which detailed the parts of Javascript we should be using, and the parts we should throw away.

Most idiomatic Javascript code can trace its foundations back to this book.

6 years before the release of *Javascript: The Good Parts, Douglas Crockford released JSLint, the first javascript linter.*

JS Linting

Extra benefits of linting in Javascript:

- There is no compiler in JS. Linting allows us to find and fix common mistakes before deploying.
- Javascript has many *bad parts*. Linting helps us ensure we are only using the *good* parts.

ESLint is now the Javascript industry standard.

How it Works

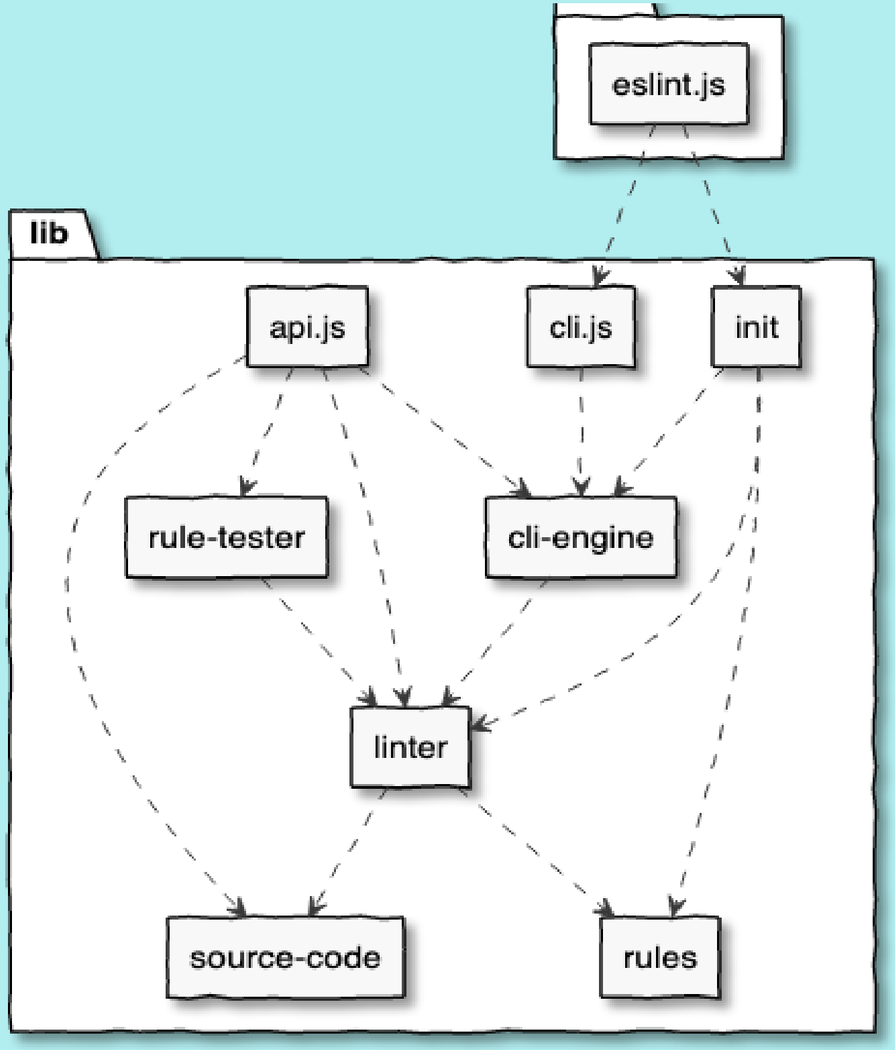
- Runs against your code.
- Has a configuration, which contains a set of rules.
- Detects and reports cases where your code violates one of the rules in the set.

How it Works

- Many rules exist but fall into two general categories:
  - Possible errors
  - Best practices + styling enforcement.
- Rules can be configured with different parameters and can be set to report as a warning or error.
- Runs as a command, or can be set to 'watch' changes in your code. When a change is made on the file system, it will automatically lint results for you.

How it Works (Really)

1. The linter is provided code as a raw string.
2. The linter parses the code and generates an AST (*Abstract Syntax Tree*) which represents the program's structure.
3. Each node in the AST represents a block of code, and is appended with the line number and column number the code starts and ends.
4. The linting configuration defines a ruleset, which is a list of rules.
5. Each rule is a function that acts on the AST. It inspects the AST for patterns that 'break' the rule. If it finds any, it reports them.



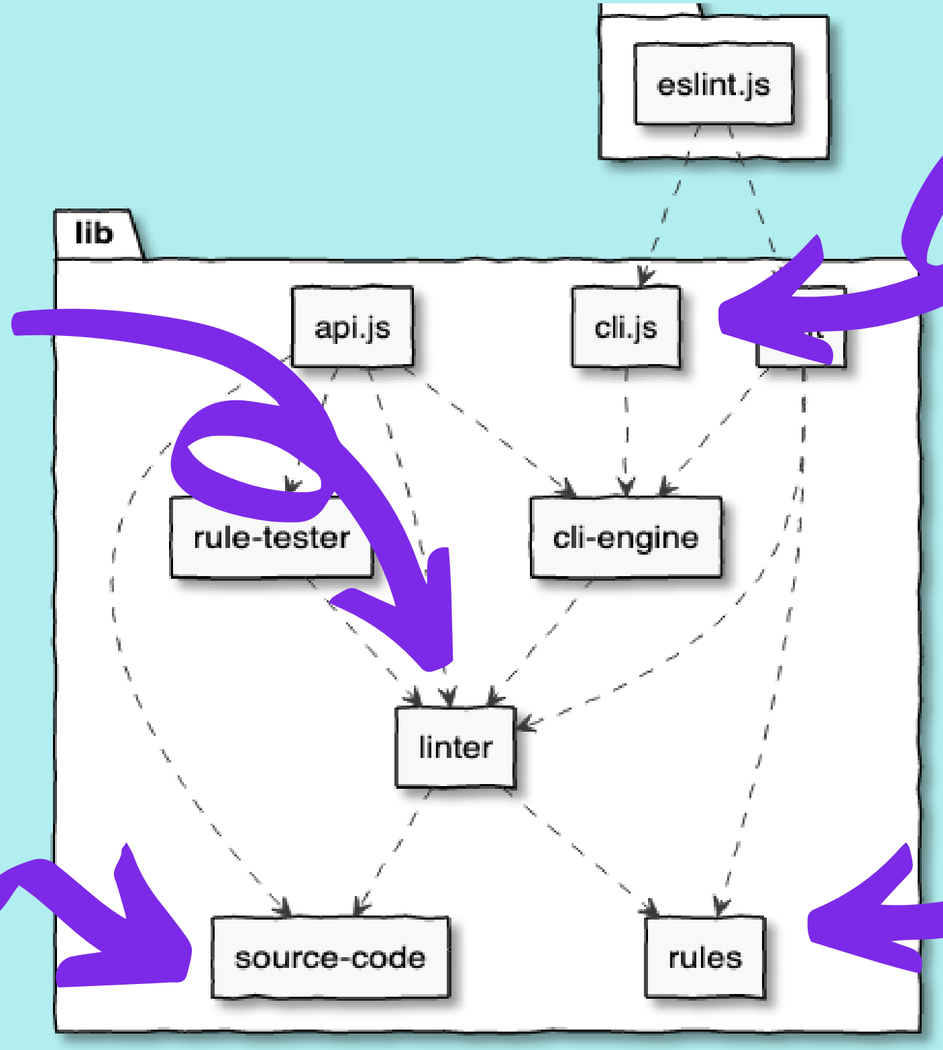
https://eslint.org/docs/developer-guide/architecture

User interface

Handles the execution of rules

Defines the rules

Represents the parsed source code



https://eslint.org/docs/developer-guide/architecture

Example Lint Rules

No Unreachable: https://eslint.org/docs/rules/no-unreachable
Stops you from writing code that cannot be reached, ie code after a return statement.

No Unused Variables: https://eslint.org/docs/rules/no-unused-vars
Stops you from defining a variable and then not using it.

No Use Before Define: https://eslint.org/docs/rules/no-use-before-define
Stops you from referencing a variable before it is defined.

Example Lint Rules

Indent: https://eslint.org/docs/rules/no-unreachable
Enforces consistent indentation.

No Var: https://eslint.org/docs/rules/no-var
Stops you from using the var keyword, enforces use of let and const.

No Console: https://eslint.org/docs/rules/no-console
Stops you from using methods on console.

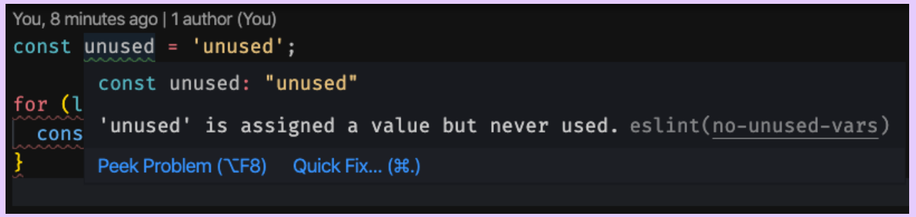
Getting started with linting

Q

How do I get started with linting?

A

You already have! **Create-React-App comes bundled with a pre-configured installation of ESLint.**



Demo

Good Work!

COMP6080
Web Front-End Programming

Week 5.1
React Hooks - useContext

Managing State beyond a Component

We've learned how to manage state within a component. However, what happens if we want to manage state between components? We have three key options:

1. Pass props down to subsequent components with "useState"
2. Use ReactJS "useContext" hook
3. Use a state management tool like **mobx** or **redux**

(3) is too complicated for the scope of this course. So we will discuss (1) and (2)

Passing props down

Let's do a demo.

useContext hook

There are a large number of complex ways to use the useContext hook. We will demonstrate a very straightforward example through a demo that you can use in your own projects.

We will use this guide.

Course Updates: Notices

- Assignment 3 has been released, draft finalised tonight
- Assignment 2 marks intend to be released next Thursday
- Course content still not overdue, but also still delayed

Course Updates: Content for later weeks

The following are content taught in later weeks, so we won't touch today:

- Building your ReactJS app (for production)
- SASS
- Demonstrating people's ass2
- CSS for React, not in a single CSS file

Course Updates: Improvements

The following are areas we're looking to improve on or take note of:

- Asking tutors to setup lab marking more quickly
- Feedback passed on to tutors about how to run help sessions and labs better - so it's been heard
- More consistent recording quality (audio/video)
  - Student with long feedback, please email me!

Course Updates: Hard to fix

The following feedback items are hard to action

- Weighting assignment 3 more towards design/UX
- Using zoom instead of blackboard (zoom more stable)
- More consistent guest lecturer format
- More linear approach to teaching
- Teaching of JQuery in the course

Course Updates: 2021 Improvements

The following feedback items are being dealt with 2021

- Course will not have the same lab marking next year
- Less lengthy workload at the start of the course
- Covering CSS grids earlier on
- (Potentially) Longer but fewer help sessions (will poll)
- (Potentially) Reducing overall workload (will poll)
- Zain's lectures were some of the highest quality, but hardest to understand, will refine

Course Updates: Unsure what to do

The following feedback I (Hayden) am unsure what to do with:

- "Some lectures from external sources"
- "course coherence"
- "More relevant teaching for the completion of lab exercises"
- "Better use of livestreamed lectures, e.g. pre-planned tutorial-like exercises."
- "I think that the assignments/labs could be a bit more refined - they totally end up being refined, but this usually happens after the initial release"
- "The assignments feels like a massive step up from labs (especially ass2) so maybe a bit more helpful material or tutorial exercise will be appreciated"

Express Servers

The most common way to build an HTTP server with NodeJS (Javascript) is to build an express server. Let's do this with NPM.

Using a limited knowledge of HTTP, combined with your knowledge of Javascript so far, this can be a very simple library to make sense of.

Starting a ReactJS application

Now we're going to start a ReactJS application inside this repository. We'll use **create-react-app** for this, which will by default use **yarn** as the package manager.

Building Components, Talking to the API

A common design pattern for ReactJS apps is to use **react-router-dom** to manage your page routes, and then have two key folders in the `src` directory:

- "/components/": For all components that don't map directly to routes
- "/pages/": For all components that map directly to routes

Let's explore this together.

Using Libraries in ReactJS

One of the best aspects about working on these ReactJS projects is that you can easily leverage countless node packages. It is VERY rare that you will have to reinvest the wheel.

A simple example for assignment 3:

- Trying to setup a countdown timer

Assignment 3

Let's explore this together.

COMP6080

Single Page Apps

Presented by
Christian Scott

Canva

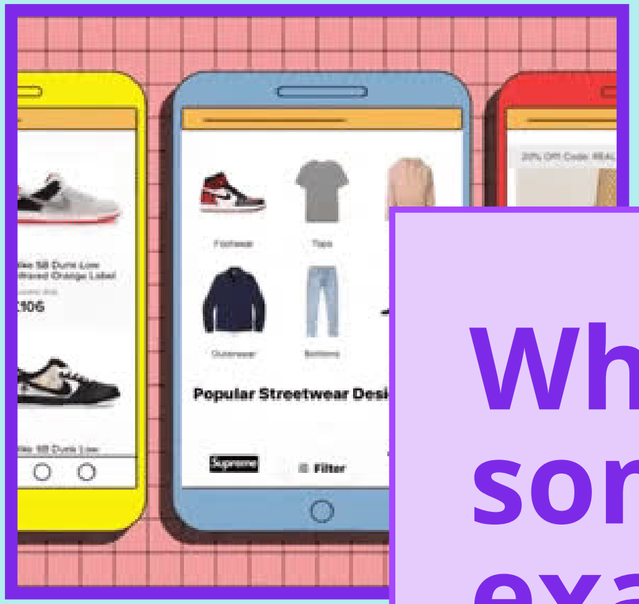
What are SPAs?

Single page applications (SPAs) are a kind of web application. They get their name from the fact that they only download a *single* HTML document.

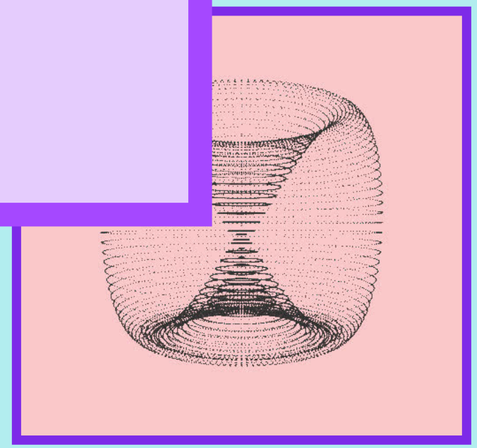
When you interact with an SPA, the content on the screen is updated via JavaScript.

This is in contrast with a traditional, multi-page website, where the content will be updated by downloading a new HTML document every time you navigate to a new page.

In an SPA, new content is downloaded over the network using JavaScript APIs like XMLHttpRequest and fetch.



What are some examples?



Single

The new versions of Reddit and Facebook are both SPAs

Multi

Sites like Wikipedia and Amazon are not SPAs

Why an SPA?

SPAs offer faster "post-load" performance that can make them feel more like a native application.

That is, after the page is loaded updates to the page are much faster since you don't need to download a new document every time you navigate to a new page.

Instead, these applications load small amounts of data from the server and use that to update the page.

By pre-fetching data or just having all the data available on first load, you can even do near-instantaneous transitions between pages.

Why *not* an SPA?

SPAs are not a silver bullet. In fact, they're quite hard to get right.

There are a few big challenges that you'll run into when building an SPA:

- Search engine optimisation (SEO)
- Performance
- Browser support

SEO

For many websites, getting a good ranking on sites like google and bing is critical for their success.

SEO is a key part of this. In short, SEO means making sure that search engine crawlers are able to *understand the content of your website.* People often pull some dirty tricks to try and boost their ranking, but most of the time it's just about making sure your content is good & that it's structured in a way that makes sense.

There are a few ways that SPAs and crawlers don't mix:

- By default, the HTML document for an SPA is empty
- The "pages" of your website aren't real
- *TODO: expand on these points*

Performance

SPAs are a double edged sword when it comes to performance.

On one hand, **once your application is loaded it will usually be a lot snappier than a traditional multi-page website**. This is because the content on the page is updated using JavaScript that you've already downloaded, rather than waiting for a new HTML document.

On the other hand, the **performance from the perspective of the initial load is usually much worse with an SPA**. This is due to two factors:

1. Downloading the JS bundle required to run an SPA is expensive
2. Not only do you need to wait for the JS bundle to download, you also need to wait for it to be parsed, compiled (yes, compiled!) and run

Performance (cont)

All this means that while your application might be snappy while it's running, your users will have to wait a bit longer to use it than they would if you had built a multi-page website instead.

However, that advantage is assuming that your users are on fast devices too! Your website might be a delight on your brand new macbook, but what's the experience like on a 5 year old, mid-range Android phone?

Not only will the initial performance be worse due to the fact that older devices will take longer to parse, compile and run your app, the performance as your app is running will be worse too – it might be so heavyweight that it slows to a halt.

Browser Support

Browsers are not designed to support SPAs out of the box. Support for SPAs is getting better all the time – but often you end up needing to rebuild basic browser functionality inside JavaScript. This re-built functionality is often incorrect and slower than the version that the browser includes.

Take navigation for example. Having a history of all the pages you have visited is very useful. Imagine if you couldn't use the back button while navigating Facebook!

How are SPAs built?

You can build an SPA using any JavaScript framework – or even without a framework at all if you feel like getting stuck in to it!

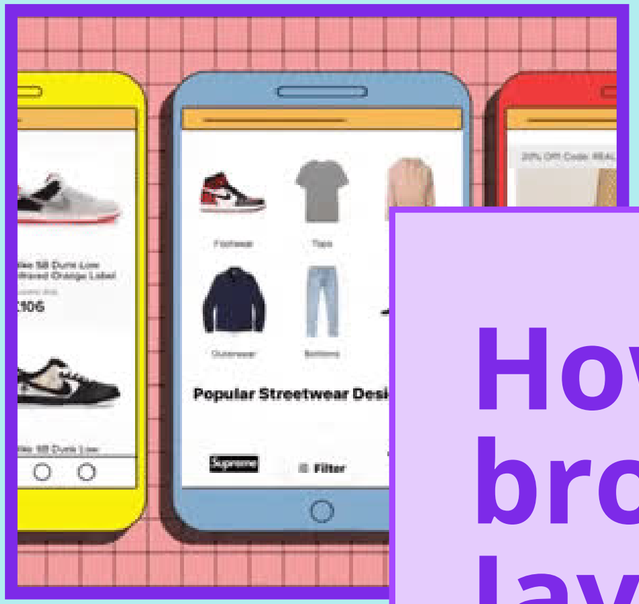
The most popular JS framework at the moment is (surprise surprise) React.JS.

For routing, the most popular library is react-router.

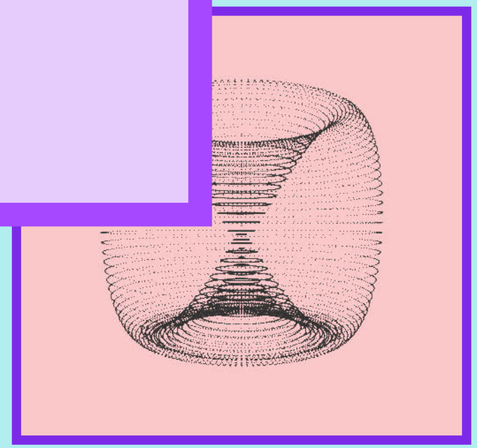
How does routing work?

TODO: finish this page

The end!



How does the browser run JavaScript?



Will the heading change?

```
<body>
  <script>
    const heading = document.querySelector('h1');
    if (heading !== null) {
      heading.textContent = 'I changed!';
    }
  </script>
  <h1>I did not change</h1>
</body>
```

Will the heading change?

No.

```
<body>
  <script>
    const heading = document.querySelector('h1');
    if (heading !== null) {
      heading.textContent = 'I changed!';
    }
  </script>
  <h1>I did not change</h1>
</body>
```

Will the heading change?

```
<body>
  <h1>I did not change</h1>
  <script>
    const heading = document.querySelector('h1');
    if (heading !== null) {
      heading.textContent = 'I changed!';
    }
  </script>
</body>
```

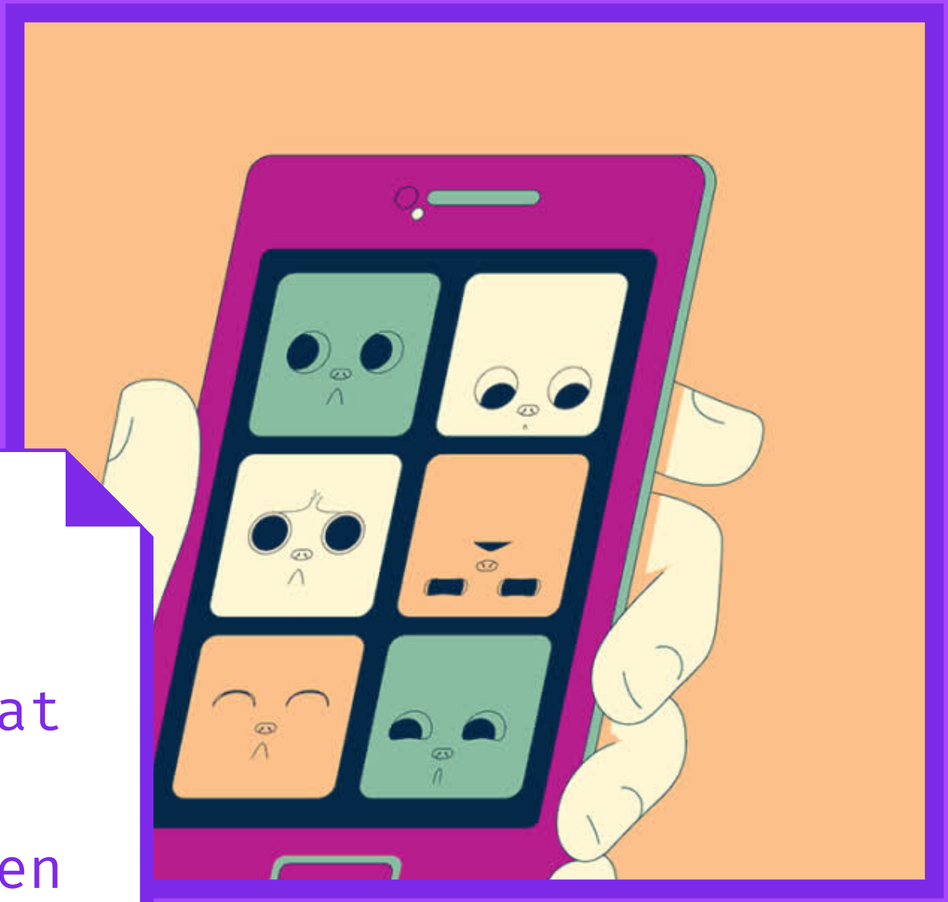
Will the heading change?

Yes!

```
<body>
  <h1>I did not change</h1>
  <script>
    const heading = document.querySelector('h1');
    if (heading !== null) {
      heading.textContent = 'I changed!';
    }
  </script>
</body>
```

Why?

JS is executed by the browser when it is encountered. This means that in the first case, the heading element has not even been created yet!



Will the heading change?

```
<body>
  <script src="change-heading.js"></script>
  <h1>I did not change</h1>
</body>
```

Will the heading change?

No.

```
<body>
  <script src="change-heading.js"></script>
  <h1>I did not change</h1>
</body>
```

Will the heading change?

```
<body>
  <h1>I did not change</h1>
  <script src="change-heading.js"></script>
</body>
```

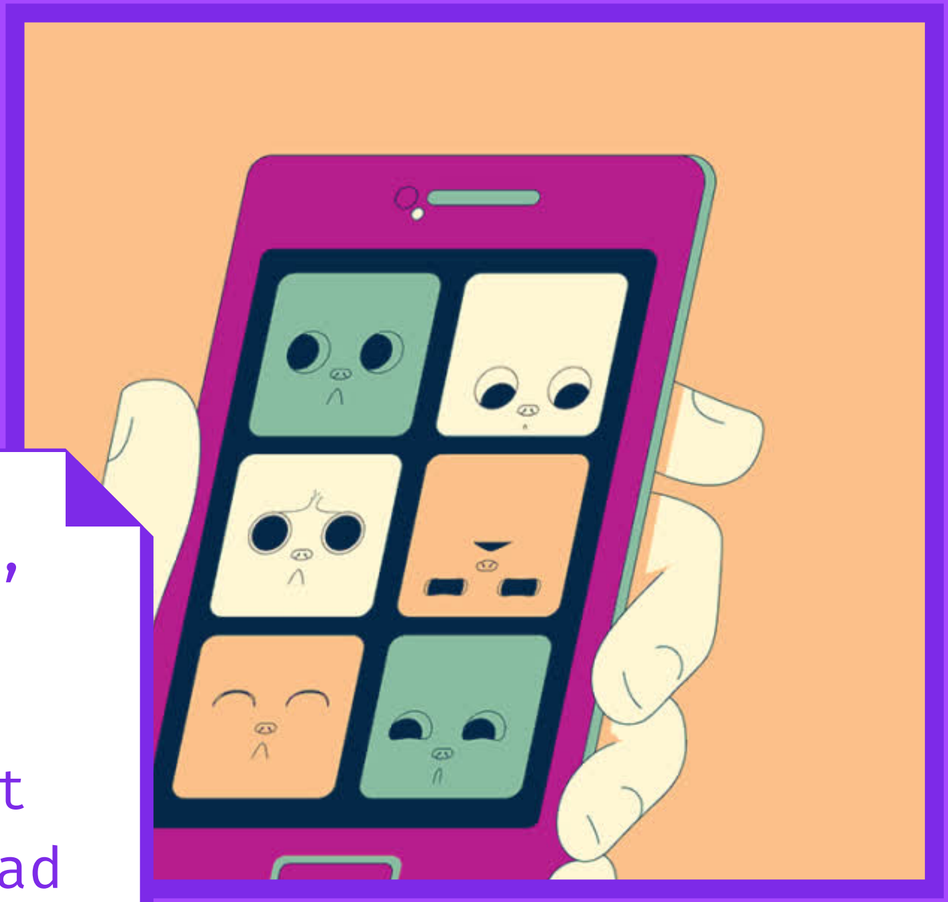
Will the heading change?

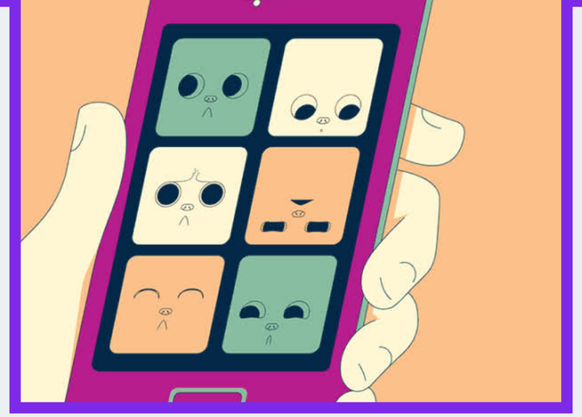
Yes!

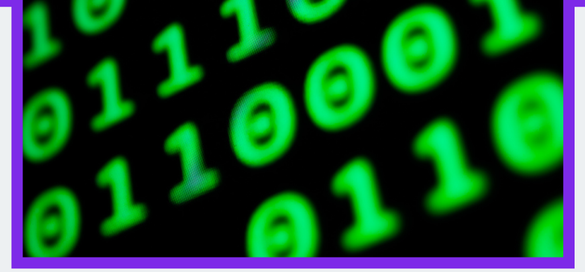
```
<body>
  <h1>I did not change</h1>
  <script src="change-heading.js"></script>
</body>
```

Why?

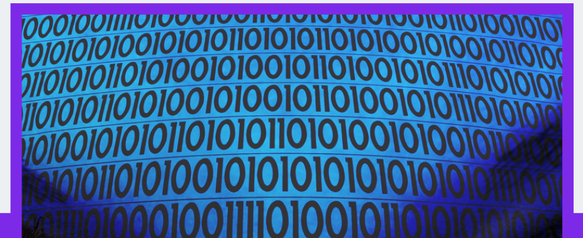
Same as before! By default, the browser will execute JavaScript when it encounters it, even if that means waiting for it to load over the internet







What if the internet is really slow?



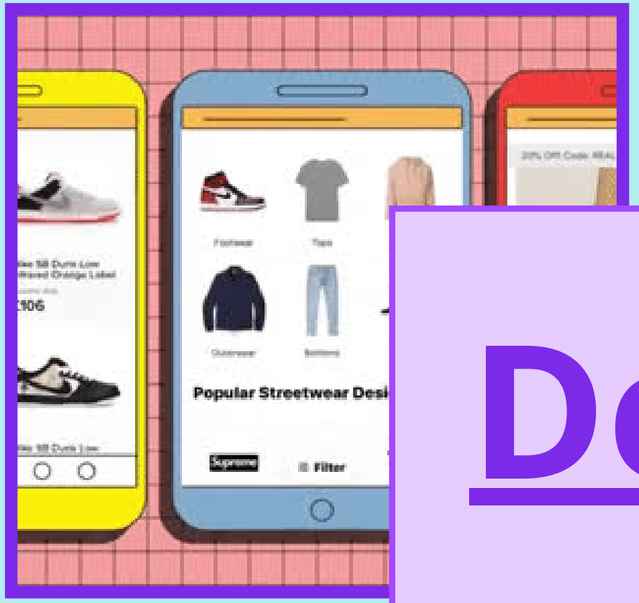
WTF is render blocking?

Before the browser can render the HTML document on your screen, it needs to *parse* the whole document. As mentioned previously, the parser will stop & execute a script whenever it encounters one.

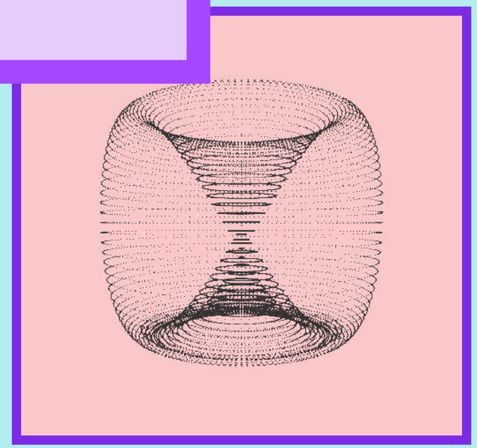
If the script is external (i.e not embedded in the document), this means that the parser will need wait for the request to complete before it can continue.

The effect of this is that sometimes you will need to wait for several seconds to see anything on the page, even once the initial request has completed! This also applies to CSS.

Google Web Fundamentals - Adding Interactivity With JavaScript



Demo!



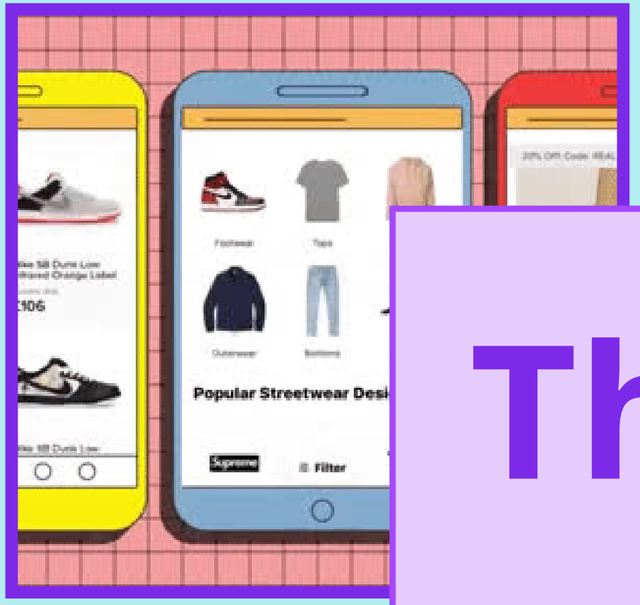
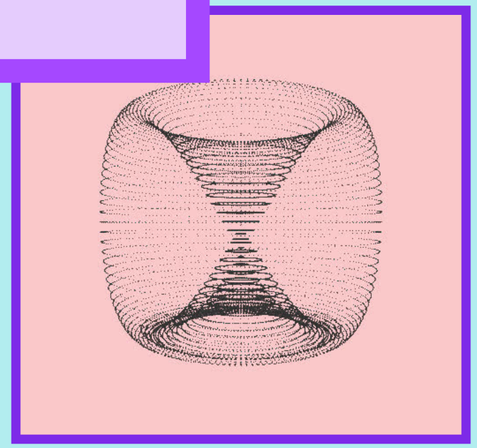
server.js — js-in-the-browser

server.js | before.html | after.html | load-items.js

delayed > server.js > ...

```javascript
const express = require("express");
const app = express();

app.use(delay(2000));

app.use(express.static("."));

app.listen(3000);

function delay(ms) {
  return (req, res, next) => {
    setTimeout(() => next(), ms);
  };
}
```

0 0 Live Share Ln 15, Col 1 Spaces: 2 UTF-8 LF JavaScript Prettier: ✓



Thanks!